

Présentation du projet :

C'est une simulation d'un petit village dans l'époque médiévale modélisant une économie locale avec des chaînes de production simplifiées. Le système simule le comportement des villageois qui effectuent des actions toutes les heures (routine dans villageois.cpp) en lien avec les bâtiments.

Tout d'abord, nous avons utilisé SFML pour l'interface graphique mais ensuite nous avons changé pour utiliser Qt à cause de nombreux problèmes liés aux librairies externes.

NOTE : L'exé est déjà compilé. Exécuter soit le raccourci dans le dossier codeVillage, ou l'exé dans codeVillage/build.

Présentation des classes :

Classe Village :

La classe village est le moteur de tout le programme. Elle est composée des bâtiments et des villageois, et elle gère le temps (les cycles/ticks), l'économie, et les actions de la population. Chaque tick informe les composants du village de l'heure actuelle. Nous avons également implémenté une fonction permettant de sauvegarder l'état du village actuel et une fonction pour importer un villageois. Les attributs principaux sont:

```
std::vector<Batiment*> batiments
std::vector<Villageois*> population
int tickActuel
double tresorVillage
```

Importation:

```
Format fichier importVillageois.txt :
10
Jean le Forgeron
Forge `
```

Classe Bâtiment :

La classe abstraite Bâtiment représente les moyens de production, de consommation, et leur attribue un inventaire de ressources. Ils peuvent contenir des villageois. Le mécanisme principal permettant de les faire fonctionner est la fonction `simulerCycle()` qui consomme ou produit des Ressources selon l'état du Bâtiment. Chaque bâtiment symbolise un maillon de la chaîne de production. Les méthodes principales sont:

```
virtual void produire();  
virtual void consommer();  
void mettreAJourStatut();
```

Classes qui héritent de Bâtiment :

- Abbaye, Boulangerie, Champ, Forge, Manoir, Mine, Moulin, Pâturage, Taverne

Classe Villageois :

Chaque villageois a un lieu de travail et un accès à la taverne (mais ils sont sans domicile parce que les temps sont difficiles). Ils suivent une routine donnée, en fonction de l'heure informée par le village, la classe vérifie que leur comportement soit en adéquation. Leur routine dépend de leur profession, par exemple le tavernier ne picole pas avec les autres à la taverne, il leur sert des boissons. Les villageois ont également un système de livraisons vers d'autres bâtiments lorsqu'ils dépassent un certain seuil de ressources dans leur inventaire. (Par exemple, un fermier ira livrer son blé au meunier)

Les méthodes principales sont:

```
StatutVillageois getStatut() const;  
void faireAction(int heure);  
void travailler();
```

Classe Ressource :

Les villageois et les bâtiments peuvent en acquérir dans leur inventaire. Dans la routine des villageois selon leur profession, ils en obtiennent plusieurs de types variés.

Attributs principaux:

```
nom : (Blé, Pain, Acier, Manuscrits...)
quantite
prixUnitaire : Valeur d'échange
nomUnite` : (kg, pièces, sacs)
```

Classe Logger :

Cette classe sert à remplacer les `std::cout` par des messages log catégorisés afin de les afficher dans le log (QTextEdit).

Classe Mainwindow :

Cette classe sert à créer des instances de nos objets (Village, Bâtiments & Villageois), c'est également ici où nous gérons l'interface graphique (nous aurions aimé que ce soit plus séparé mais on n'a pas eu le temps).

Relation entre les classes :

Champ, Abbaye, Mine, Pâturage, Moulin, Boulangerie, Taverne, Forge, Manoir héritent de Bâtiment (*héritage*)

Exemples divers:

Village contient `std::vector<Bâtiment*>` et `std::vector<Villageois*>` (*agrégation*)

Village 1 ◇—— 0..* Bâtiment

- Bâtiment contient `std::map<std::string, Ressource*>` (*composition*)

Bâtiment 1 ◆—— 0..* Ressource

-Villageois a des pointeurs vers Bâtiment (*association*)

Un villageois connaît son lieu de travail et peut interagir avec.

Villageois 0..* — 1 Bâtiment

Respect du Cahier des Charges :

Spécificateurs d'accès (public, privé, protégé):

Exemple :

- Ressource.hpp:
 - `private`: attributs (nom, quantite, prixUnitaire, nomUnite)
 - `public`: méthodes (constructeur, getters, setters)
- Bâtiment utilise `protected`: pour les classes dérivées

Comparaisons entre objets

Dans ressource.hpp

```
bool operator==(const Ressource& autre) const;
```

```
bool operator>(const Ressource& autre) const;
```

Méthodes virtuelles

Présentes entre autres dans Bâtiment.hpp

```
virtual void produire() = 0;  
virtual void consommer() = 0;  
virtual std::string getType() const = 0;  
virtual void afficherInfos() const = 0;
```

Implémentation:

- `Champ::produire()`, `Abbaye::produire()`
- `Champ::getType()`, `Mine::getType()`, etc.

Opérateurs

```
bool operator==(const Ressource& autre) const; // comparaison  
d'identité  
bool operator>(const Ressource& autre) const; // comparaison de  
quantité
```

Lecture/écriture de fichiers

```
void Village::sauvegarder(const std::string& nomFichier)
```

Cette méthode prend un fichier (en l'occurrence datatext.txt) et enregistre des informations (jour, heure, trésor, prospérité, habitants).

```
bool Village::importerVillageois(const std::string& nomFichier)
```

Cette méthode prend un fichier texte de trois lignes (la première correspond à l'id du villageois, la deuxième à son nom et la troisième à son lieu de travail), lit ces informations et crée le villageois importé.

 DataVillage.txt	23-Dec-25 15:09
 importVillageois.txt	23-Dec-25 15:21

(Dans le fichier build)

Pointeurs, const, transmission par valeur/référence

Pointeurs:

```
Batiment* lieuTravail; // association (ne possède pas l'objet)  
std::vector<Villageois*> population; // agrégation
```

Méthodes const:

```
std::string getNom() const; // ne modifie pas l'objet  
int getQuantiteRessource(const std::string& nomRessource) const;
```

Transmission par référence:

```
void setNom(const std::string& n); // const ref pour éviter  
copie  
bool ajouterOccupant(Villageois* v); // pointeur pour modifier  
l'objet
```

Transmission par valeur:

```
void ajouterRessource(const std::string& nomRes, int quantite);  
// int passé par valeur
```

Difficultés rencontrées :

Migration SFML à Qt, refonte complète de l'interface avec Qt

Gestion mémoire, risques de fuites mémoire et segfaults

Logique métier mélangée à l'interface (manque de temps pour réécriture complète)

Filtres checkbox qui ne permettent pas de voir plus de deux types d'infos en même temps

Voies d'amélioration :

Icônes de production sur les bâtiments

Un log moins chargé

Comportement autonome pour les villageois

Économie plus complexe

Évènements (peste, guerre, récoltes abondantes, etc)

Utilisation des outils d'IA :

Nous nous sommes servis parfois (à contrecoeur) de l'intelligence artificielle Claude mais nous avons constaté que ce n'était pas conçu pour les projets de ce type puisque ses suggestions cassaient souvent notre architecture existante et que c'était souvent beaucoup plus lent que d'écrire sans assistance.

Cependant, ça a été assez utile lors du débogage. On a donc choisi de tout écrire à la main en se servant de la documentation C++, stackoverflow, geeks4geeks, ainsi que cette [playlist](#) de tutoriels Qt sur youtube.

Diagramme de Classes

>> voir Diagramme.pdf